The Journal of Open Source Software

geeagri: A Python package for agricultural analysis with Google Earth Engine

Krishnagopal Halder [1¶], **Amit Kumar Srivastava**[1], **Manmeet Singh**[2], and **Frank Ewert** [1,3]

1 Leibniz Centre for Agricultural Landscape Research (ZALF), Eberswalder Str. 84, 15374 Müncheberg, Germany **2** Department of Earth and Planetary Sciences, Jackson School of Geosciences, University of Texas, Austin, USA **3** Institute of Crop Science and Resource Conservation, University of Bonn, Katzenburgweg 5, D-53115, Bonn, Germany ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 03 November 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

geeagri is a Python package designed for advanced agricultural monitoring and analysis using Google Earth Engine (GEE). GEE is a powerful cloud-computing platform with a multi-petabyte catalog of satellite imagery and geospatial datasets (Gorelick et al., 2017). Although GEE provides its robust JavaScript and Python APIs, and packages like geemap have improved its interactive mapping capabilities (Wu, 2020), researchers in domain-specific fields like agriculture still face significant challenges. Tasks such as crop modeling (Ines et al., 2013) and machine learning (Zhu et al., 2017) often demand complex, repetitive workflows for geospatial data collection and extensive pre-processing before analysis.

geeagri addresses these challenges by providing high-level, user-friendly workflows for agricultural analysis. It encapsulates data collection and processing pipelines within an object-oriented framework, enabling both simple and advanced analyses to be performed seamlessly on GEE. By automating repetitive tasks and standardizing data processing procedures, geeagri facilitates the work of crop modelers, remote sensing analysts, and machine learning researchers, thereby reducing computational overhead on local systems and minimizing the need for extensive manual scripting.

Statement of need

Agricultural research increasingly relies on satellite-derived and climate data for applications such as crop modeling, data assimilation, yield forecasting, and vegetation monitoring (Ines et al., 2013). While Google Earth Engine (GEE) offers vast geospatial datasets and powerful cloud-based computation (Gorelick et al., 2017), its standard JavaScript and Python APIs require extensive manual scripting to perform domain-specific tasks. Existing tools like geemap enhance visualization and general geospatial analysis (Wu, 2020), but they do not fully address the specialized needs of agricultural workflows.

geeagri addresses this gap by providing high-level, domain-focused workflows for agricultural analysis within GEE. It streamlines tasks such as large scale data extraction (tabular or imagery), cloud masking, curve fitting, and other complex processing pipelines through an object-oriented and modular framework. The package's functionality is targeted at specific user groups: - **Crop modelers**, who require high-temporal-resolution climate data (e.g., temperature, precipitation, solar radiation) and phenological indicators at site or regional scales (Ines et al., 2013). - **Remote sensing analysts**, who need seamless workflows for data extraction and data preprocessing tasks such as cloud masking and advanced analyses including Principal Component Analysis and Harmonic Regression (Beck et al., 2006). - **Machine learning researchers** training vision

models on large number of tile-based image data from GEE (Zhu et al., 2017).

Key Features

- **Parallelized Data Extraction:** Automates the retrieval of satellite time series and climate datasets for any point or polygon location. Supports efficient parallel downloading of image chips (e.g., Sentinel-2 (Drusch et al., 2012), Landsat (Wulder et al., 2016)) for large-scale machine learning applications.
- **Automated Cloud Masking and Filtering:** Provides user-defined cloud thresholding and masking with minimal coding effort.
- **Synthetic Time Series Generation:** Creates temporally consistent datasets (e.g., 5-day or 10-day intervals) from irregular satellite observations to support time-series analyses (Chen et al., 2004).
- **Phenological Analysis:** Performs curve fitting and extracts key land surface phenology metrics, including the start (SOS), end (EOS), and peak (POS) of the growing season (Jönsson & Eklundh, 2004).
- **High-Level Workflow Abstraction:** Encapsulates multi-step GEE processes into simple functions and object-oriented classes, enabling reproducible and streamlined agricultural data workflows.

Example Use

The example below demonstrates how `geeagri` can be used to extract daily climate time series (temperature, precipitation, and solar radiation) from the ERA5-Land dataset for a specific point location (see **??**).

```python
import ee
import geeagri
from geeagri.extract import extract_timeseries_to_point

# Authenticate and initialize the Earth Engine API
ee.Authenticate()
ee.Initialize()

# Define point location (longitude, latitude)
lon, lat = -98.15, 30.50
point = ee.Geometry.Point([lon, lat])

# Load ERA5-Land daily aggregated climate dataset
era5_land = ee.ImageCollection("ECMWF/ERA5_LAND/DAILY_AGGR")

# Extract daily temperature, precipitation, and solar radiation time series
era5_land_point_ts = extract_timeseries_to_point(
    lat=lat,
    lon=lon,
    image_collection=era5_land,
    start_date="2020-01-01",
    end_date="2021-01-01",
    band_names=[
        "temperature_2m_min",
        "temperature_2m_max",
        "total_precipitation_sum",
        "surface_solar_radiation_downwards_sum",
```

DRAFT

```
    ],
    scale=11132,  # spatial resolution in meters (~11 km)
)
```

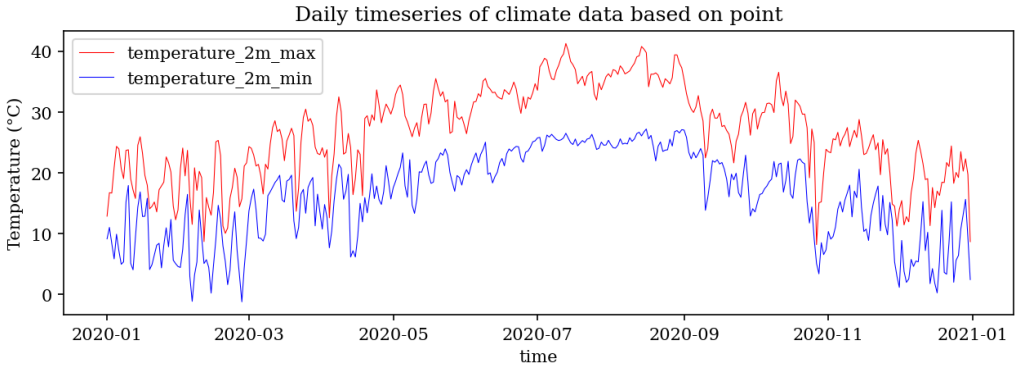

Figure 1: Example of extracting a daily climate time series from ERA5-Land for a single point using geeagri. The plot shows daily maximum and minimum temperature values for 2020, illustrating how the extracted data can support crop model simulations and environmental analyses.

geeagri Tutorials

Comprehensive tutorials, example notebooks, and API documentation are available to help users get started with geeagri and explore its functionality:

- **Example Notebooks**: step-by-step demonstrations of common workflows.
- **API Documentation and Website**: detailed reference of available modules, classes, and functions, with usage guidelines and examples.

Acknowledgements

We gratefully acknowledge the foundational work of the Google Earth Engine team (Gorelick et al., 2017) and the geemap package (Wu, 2020), which form the basis of geeagri. This work has been supported by the German Federal Ministry of Education and Research (BMBF) through the projects LL-SYSTAIN (Grant Labs-2024-IÖR) and FAIRagro (Grant NFDI 51/1), and by the Biotechnology and Biological Sciences Research Council (BBSRC) through the project SynPAI (Grant BB/S020969).

References

Beck, P. S., Atzberger, C., Høgda, K. A., Johansen, B., & Skidmore, A. K. (2006). Improved monitoring of vegetation dynamics at very high latitudes: A new method using MODIS NDVI. *Remote Sensing of Environment*, *100*(3), 321–334.

Chen, J., Jönsson, P., Tamura, M., Gu, Z., Matsushita, B., & Eklundh, L. (2004). A simple method for reconstructing a high-quality NDVI time-series data set based on the savitzky–golay filter. *Remote Sensing of Environment*, *91*(3-4), 332–344.

Drusch, M., Del Bello, U., Carlier, S., Colin, O., Fernandez, V., Gascon, F., Hoersch, B., Isola, C., Laberinti, P., Martimort, P., & others. (2012). Sentinel-2: ESA's optical high-resolution mission for GMES operational services. *Remote Sensing of Environment*, *120*, 25–36.

Gorelick, N., Hancher, M., Dixon, M., Ilyushchenko, S., Thau, D., & Moore, R. (2017). Google earth engine: Planetary-scale geospatial analysis for everyone. *Remote Sensing of Environment*, *202*, 18–27.

DRAFT

Ines, A. V., Das, N. N., Hansen, J. W., & Njoku, E. G. (2013). Assimilation of remotely sensed soil moisture and vegetation with a crop simulation model for maize yield prediction. *Remote Sensing of Environment*, *138*, 149–164.

Jönsson, P., & Eklundh, L. (2004). TIMESAT—a program for analyzing time-series of satellite sensor data. *Computers & Geosciences*, *30*(8), 833–845.

Wu, Q. (2020). Geemap: A python package for interactive mapping with google earth engine. *Journal of Open Source Software*, *5*(51), 2305.

Wulder, M. A., White, J. C., Loveland, T. R., Woodcock, C. E., Belward, A. S., Cohen, W. B., Fosnight, E. A., Shaw, J., Masek, J. G., & Roy, D. P. (2016). The global landsat archive: Status, consolidation, and direction. *Remote Sensing of Environment*, *185*, 271–283.

Zhu, X. X., Tuia, D., Mou, L., Xia, G.-S., Zhang, L., Xu, F., & Fraundorfer, F. (2017). Deep learning in remote sensing: A comprehensive review and list of resources. *IEEE Geoscience and Remote Sensing Magazine*, *5*(4), 8–36.

DRAFT